第三节 开发环境、规范与测试

肖逸飞
xyf1989@uestc.edu.cn

课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. 打包解包及其关键技术
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

软件开发环境

软件开发环境

软件开发环境是指用于进行软件开发的工具、平台和设置的组合。它提供了一个开发者可以编写、测试和分析代码的环境。其通常包括：

1. **编程语言和框架**：编程语言是开发者用来编写软件的规则和语法。框架是一组提供了特定功能和结构的代码库，简化了开发过程。
2. **开发和调试工具**：包括集成开发环境、文本编辑器、编译器、连接器、调试器等。
3. **版本控制系统**：用于跟踪和管理代码的不同版本。
4. **测试和分析工具**：测试工具用于自动化测试和验证软件的功能和性能。分析工具用于评估和优化软件的性能，识别性能瓶颈和优化机会。
5. **集成和部署工具**：集成工具用于将不同的开发工具和系统集成在一起，实现自动化的构建、测试和部署流程。部署工具用于将软件应用程序部署到目标环境中，例如服务器、云平台等。

不同的开发者通常会根据自己的偏好和项目要求选择适合的软件开发环境。

软件开发环境

《实验指导书1》给出了一个使用VSCode远程连接Linux服务器进行C++语言开发的软件开发环境搭建教程，可供参考。

软件开发环境各不相同，选择合适的环境即可。但是，对于项目协作来说，软件开发环境**务必统一**，以防止出现不必要的麻烦。

统一开发环境的方法（推荐）：

1. **远程共享同一套软件开发环境**（共享固定IP的远程IDE）
2. **容器/虚拟机镜像复制**
3. **基础设施即代码**（IaC： 通过代码脚本，几分钟内就能自动创建、配置和部署一整套完整的环境（开发、测试、生产），极大提升效率。 ）

软件编码规范

除软件开发环境需要统一外，制定并统一软件开发规范同样重要：

1. 统一代码风格，有助于进行代码审查；
2. 便于代码维护；
3. 降低出现程序Bug的风险；
4. 养成良好的编程习惯；

编码规范实例：

Google - 开源项目风格指南

华为 - 软件编程规范

软件测试

基本概念

软件测试：为了**发现**软件开发过程中所发生的**错误**而执行所开发程序或程序中某个模块的过程。

据统计，在软件开发过程中，软件测试的工作量约占软件开发总工作量的**40%**左右。

软件测试的方法可根据不同的维度被分为若干类。

比如：

- 按是否运行分类
- 按测试步骤分类
- 按测试方法分类

测试用例

测试用例是由**测试数据**和**预期结果**构成的。为了发现程序中的错误，应竭力设计能暴露错误的测试用例。

一次成功的测试是发现了至今尚未发现的错误的测试。

测试用例的产生来源：1）黑盒测试方法 2）白盒测试方法

测试用例一般包括：

程序版本号	模块名称	用例编号	用例名称	用例级别	预置条件	测试输入	操作步骤	预期结果	实际输出	测试时间	测试人员

测试用例

示例

项目名称	飞机订票系统		项目编号	001	
开发人员	No_1		模块信息	登录页面	
用例作者	No_1		参考信息	需求说明、设计说明	
测试类型	功能测试	设计日期	2012 年 12 月 5 日	测试人员	***
测试方法	手工测试和自动化测试结合			测试日期	2012年12月13日
测试对象	测试用户能否正常登陆				
前置条件	存在正确的用户名：登陆页面正常装载（用户名不固定，密码为 mercury 不区分大小写）				

用例编号	操作	输入数据	预期结果	实际结果	测试状态
01	输入正确的用户名、正确的密码，按 Enter 键	用户名：mingzi，密码：mecury	正常登陆	正常登陆，转入对应的系统页面	Pass
02	输入正确的用户名，正确的密码（没区分大小写），按 Enter 键	用户名：mingzi，密码：MERTURY/mecury	正常登陆	正常登陆，转入对应的系统页面	Pass
03	输入的用户名为空，正确的密码（没区分大小写），按 Enter 键	密码：MERTURY	登陆失败	提示请输入用户名	fail
04	输入正确的用户名，密码为空，按 Enter 键	用户名：mingzi	登陆失败	提示请输入密码	fail
05	输入的用户名和密码均为空，按 Enter 键		登陆失败	提示请输入用户名	fail
06	输入错误的用户名，正确的密码（没区分大小写），按 Enter 键	用户名：zi，密码：MERTURY/mecury	登陆失败	提示请输入大于 4 个字符长度的用户名	fail



目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

测试分类

1.按是否运行被测软件

静态测试：不实际运行被测软件，而只是静态地检查程序代码和文档，来发现软件错误。

通常，代码走查，代码审计均可被看作静态测试。

动态测试：通过实际运行程序来检验软件的动态行为和运行结果的正确性。

测试分类

静态代码分析工具可以帮助检测代码中的潜在问题、漏洞、代码重复和代码覆盖率，并提供详细的报告和建议。以下是一些常用的静态代码分析工具：

Klocwork：

Klocwork是一个领先的静态代码分析工具，它有**1000多**个检查器，对症不同种类代码缺陷，同时用户可定制检查方案。它还可以提供差异分析，这个功能并不常见。

Klocwork不仅是一个静态代码分析工具，同时也是一个SAST（静态应用程序安全测试工具），几乎全面覆盖了**安全性能**方面。

Cppcheck：

Cppcheck是另一个热门的静态代码　分析工具，集开源、免费、跨平台且专用于C和C++的优势于一身。

Cppcheck最大的特点是易用性，操作简单为它吸引了一大批用户。

SonarQube：

SonarQube是一个开源的静态代码检测平台，**支持多种编程语言**，如Java、 C++、 C#、Python等。它能够检测代码中的潜在问题、漏洞、代码重复和代码覆盖率，并提供详细的报告和建议。

Coverity：

Coverity主要用于在编写代码时查找错误和弱点，为开发者节省时间。此外，Coverity还有提供了一个免费的云服务——Coverity Scan，主要针对开源社区。

测试分类

1.按是否运行被测软件

静态测试：不实际运行被测软件，而只是静态地检查程序代码和文档，来发现软件错误。

通常，代码走查，代码审计均可被看作静态测试。

动态测试：通过实际运行程序来检验软件的动态行为和运行结果的正确性。

动态测试又可根据不同的**维度**进行分类。

- 按**测试步骤**分类：单元测试、集成测试、系统测试和验收测试
- 按**测试方法**分类：黑盒测试、白盒测试



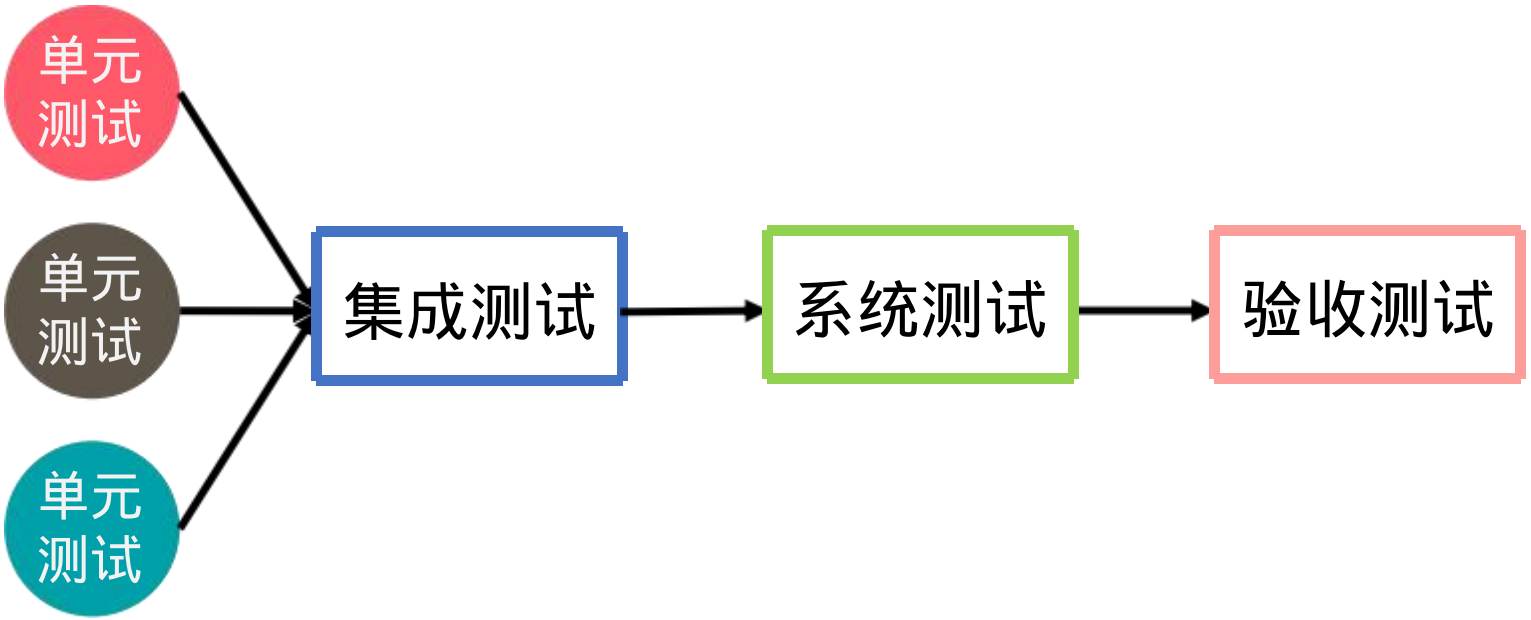
目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

测试分类

2.按测试步骤

测试的步骤按4个步骤进行，分为单元测试、 集成测试、 系统测试和验收测试。

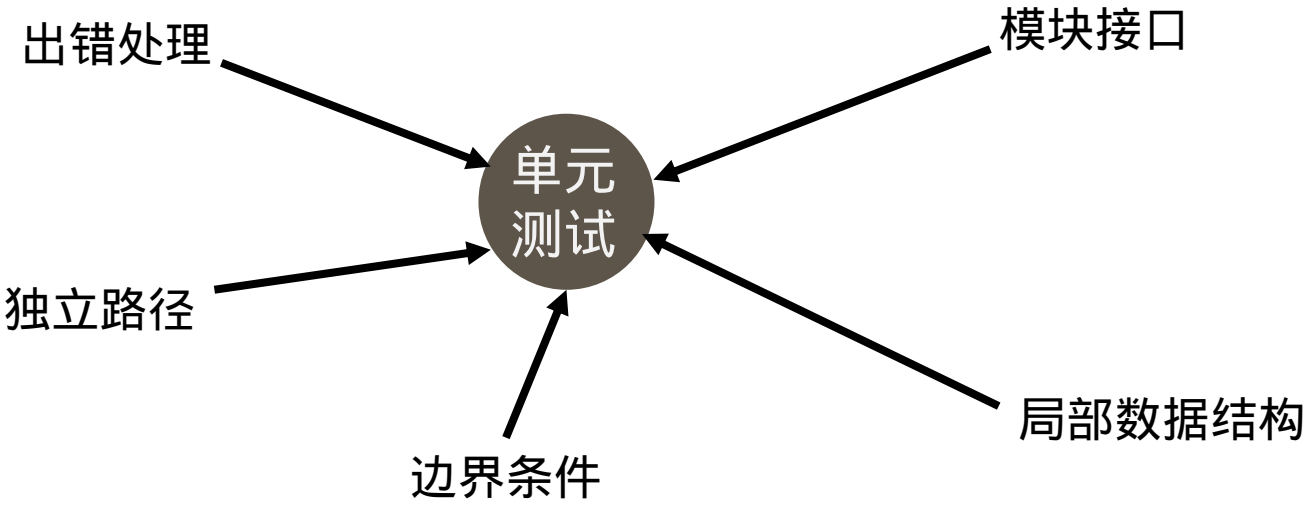


测试分类

单元测试

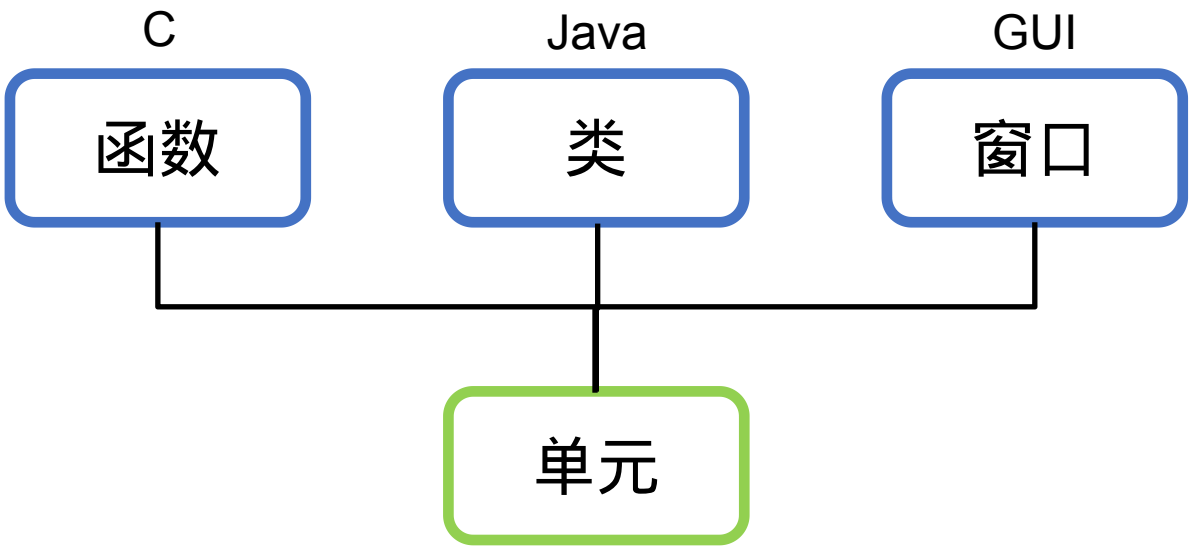
单元测试又称为**模块测试**，是针对软件设计的最小单位(模块)进行正确性检验的测试工作。
目的：检查每个程序单元能否正确实现详细设计说明中的模块功能、性能、接口和设计约束等要求，以及发现各模块内部可能存在的各种错误

单元测试一般使用白盒测试方法，测试以下内容：



测试分类

对于不同的语言，会有不同的单元，比如，C语言的单元是函数，Java的单元是类，图形化的软件的单元可以指一个窗口或一个菜单等



单元测试工具：boost test、google test。

边界条件

测试分类

集成测试

集成测试也称为**组装测试、联合测试**。它将已通过单元测试的模块集成在一起，主要测试模块之间的协作性。
从组装策略而言，可以分为2类：
1）分为一次性组装
2）增量式组装(包括自顶向下、自底向上及三明治)

敏捷开发的Scrum流派中，每一次sprint都是一次集成，集成测试一般采用**黑盒**或**灰盒**测试方法。

灰盒测试：测试人员**不直接去读/改每一行代码，但对系统的内部架构、数据库表结构、接口（API）协议、日志输出等有一定程度的了解**（像看一个半透明的灰盒子）。

测试分类

确认测试： 它的唯一依据是《软件需求规格说明书》（SRS）。测试目的是验证软件的功能、性能、接口以及其他特性的表现，是否与用户当初提出的需求完全吻合。

系统测试

系统测试，是将通过**确认测试**的软件，作为整个计算机系统的一个元素，与计算机硬件、外设、某些支持软件、数据和人员等其他系统元素结合在一起，在实际运行环境下，对计算机系统进行一系列的集成测试和确认测试。

冒烟测试、性能测试、压力测试都属于系统测试。

系统测试基本上都是**黑盒测试**。

测试分类

验收测试

验收测试与需求分析相对应，用来验证软件/服务在商业上是否满足了用户要求或市场需求，是否达到了发布的标准。

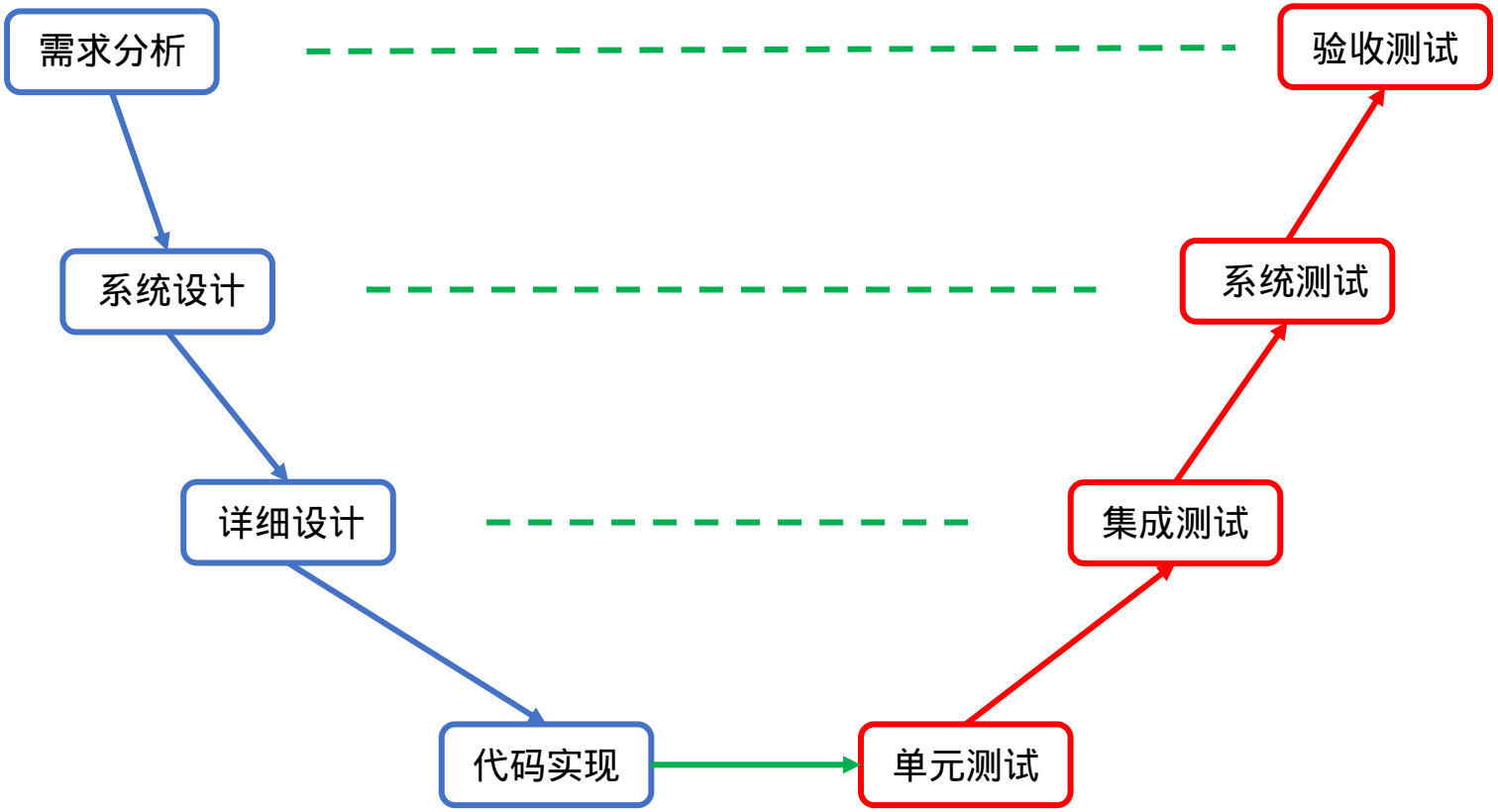
验收测试和系统测试都是针对整个软件/服务的，然而它们的区别在于：系统测试和之前的单元测试和集成测试都是在检验是否正确地制造了东西，而验收测试是在检验是否制造了正确的东西。

Doing things right VS Doing the right things

系统测试　　验收测试

验收测试基本上都是**黑盒测试**。

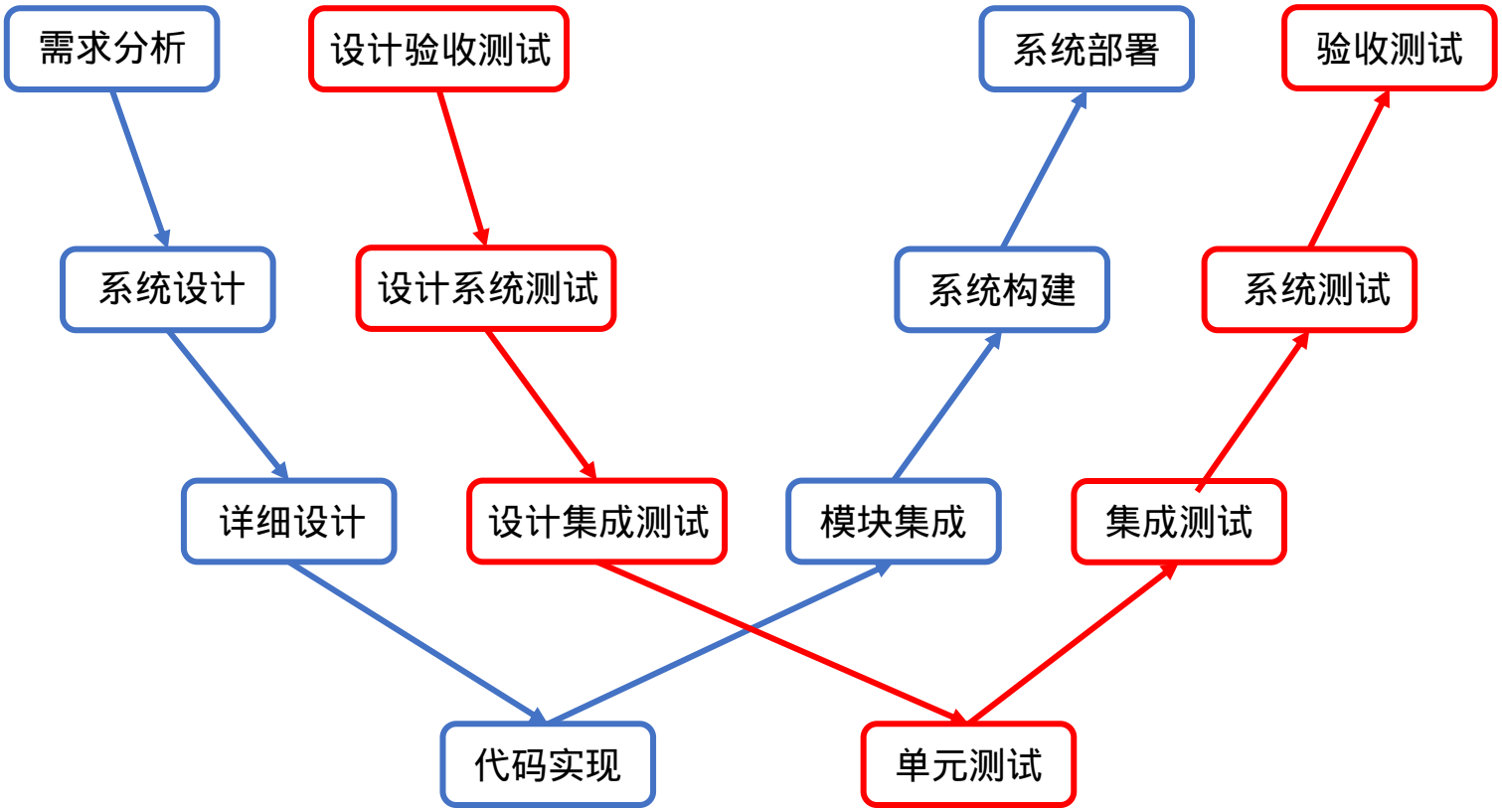
根据测试所对应的软件开发阶段



V字模型

测试驱动开发（TDD）

W字模型（V字模型的改进）



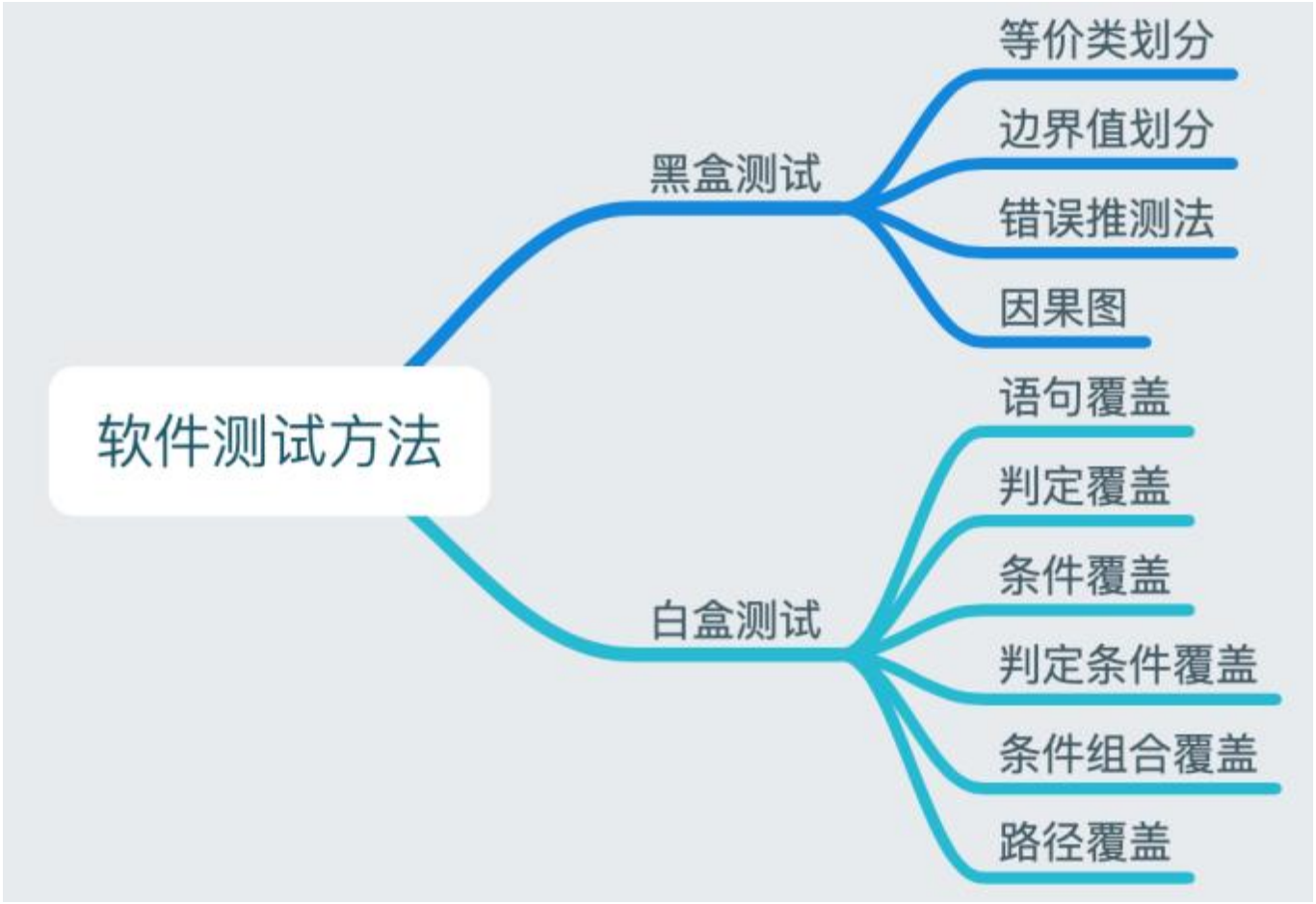


目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

测试分类

3.按测试方法


软件测试方法
黑盒测试
等价类划分
边界值划分
错误推测法
因果图
白盒测试
语句覆盖
判定覆盖
条件覆盖
判定条件覆盖
条件组合覆盖
路径覆盖

黑盒测试



等价类划分

等价类划分法是一种典型的，并且是最基础的黑盒测试用例设计方法。采用等价类划分法时，完全不用考虑程序内部结构，设计测试用例的唯一依据是软件需求规格说明书。

所谓等价类，是输入条件的一个子集合，该输入集合中的数据对于揭示程序中的错误是等价的。从每一个子集中选取少数具有代表性的数据，从而生成测试用例。

等价类划分

例：设计一个**档案管理系统**，要求用户输入以年月表示的日期。假设日期限定在1990年1月-2049年12月，并规定日期由6位数字字符组成，前4位表示年，后2位表示月。

日期检查输入条件等价类

输入条件	有效等价类	无效等价类
日期的类型及长度	（1）6位数字字符	（2）有非数字字符 （3）少于6位数字字符 （4）多于6位数字字符
年份范围	（5）在1990-2049之间	（6）小于1990 （7）大于2049
月份范围	（8）在01-12之间	（9）等于00 （10）大于12

等价类划分

设计测试用例，覆盖所有有效等价类；
再针对每一个无效等价类，设计一个测试用例。

日期检查测试用例

测试数据	期望结果	覆盖等价类
200211	有效输入	（1）（5）（8）
95June	无效输入	（2）
20036	无效输入	（3）
2001006	无效输入	（4）
198912	无效输入	（6）
205401	无效输入	（7）
200100	无效输入	（9）
200113	无效输入	（10）

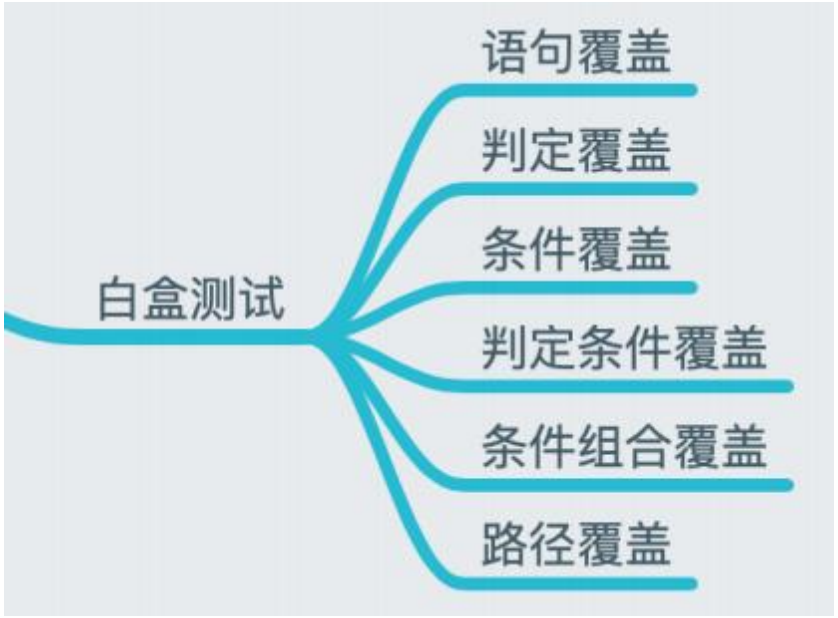
边界值分析法

边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。**通常边界值分析法是作为对等价类划分法的补充**，这种情况下，其测试用例来自等价类的边界。

采用边界值分析法设计的日期检查测试用例

测试数据	期望结果	选取理由
199001	有效输入	最小年份最小日期
199012	有效输入	最小年份最大日期
199000	无效输入	刚好小于最小日期
199013	无效输入	刚好大于最大日期
204901	有效输入	最大年份最小日期
204912	有效输入	最大年份最大日期
204900	无效输入	刚好小于最小日期
204913	无效输入	刚好大于最大日期

白盒测试



语句覆盖	每条语句至少执行一次
判定覆盖	程序中每个分支的可能值至少出现一次
条件覆盖	程序中每个判定的每个条件的各种可能值至少出现一次
判定条件覆盖	同时满足判定覆盖标准和条件覆盖标准
条件组合覆盖	每个判定中各条件的每种组合至少出现一次
路径覆盖	程序中的每条可能路径至少执行一次

白盒测试

【语句覆盖】

x=1, y=2, z=0　　TT　　sacbed

【判定覆盖】

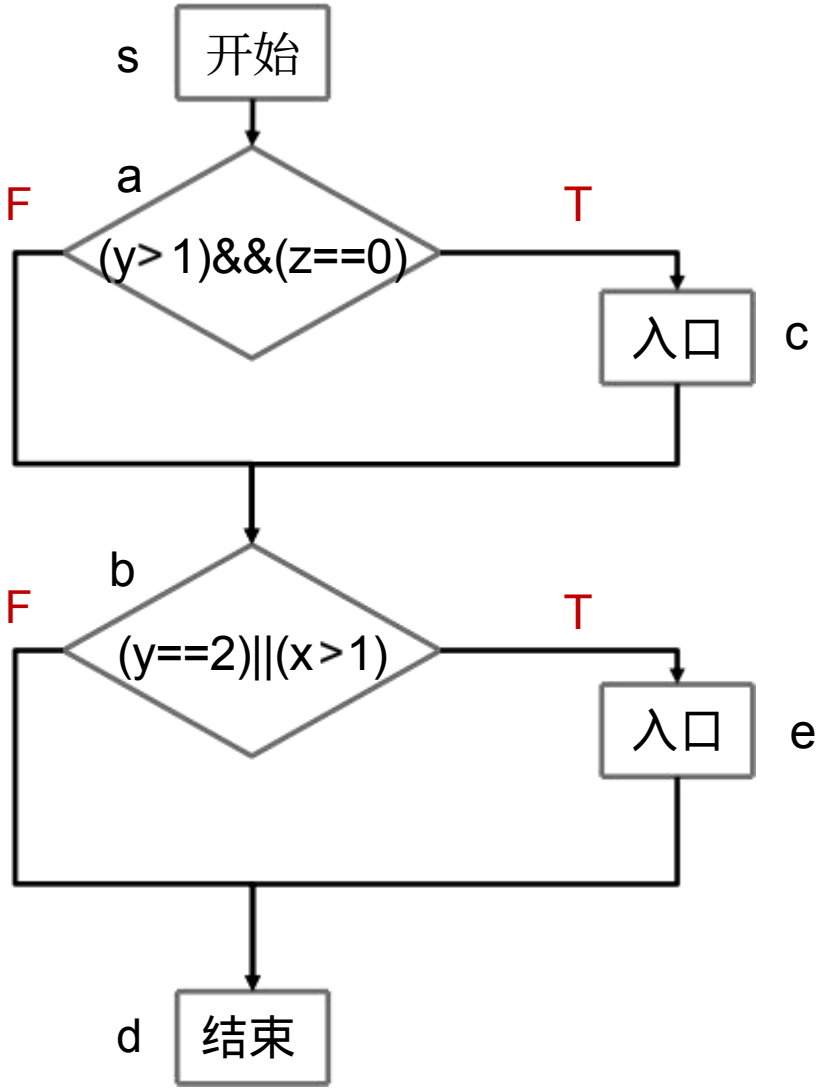
x=1, y=2, z=0　　TT

x=1, y=0, z=1　　FF

【条件覆盖】

x=1, y=2, z=0　　TTTF

x=2, y=1, z=1　　FFFT



白盒测试

【判定条件覆盖】

x=4, y=2, z=0　　TT, TTTT

x=1, y=1, z=1　　FF, FFFF

【条件组合覆盖】

x=4, y=2, z=0　　TTTT

x=1, y=2, z=1　　TFTF

x=2, y=1, z=0　　FTFT

x=1, y=1, z=1　　FFFF

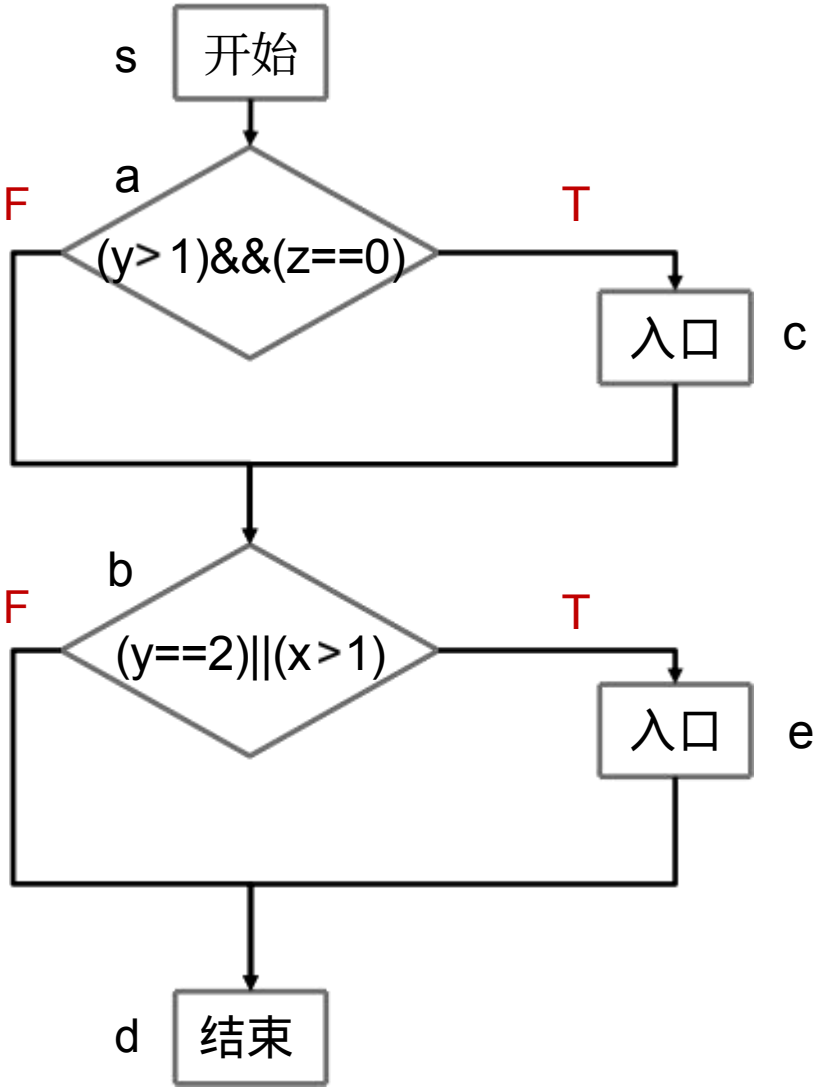
【路径覆盖】

x=4, y=2, z=0　　TT　　sacbed

x=1, y=2, z=1　　FT　　sabed

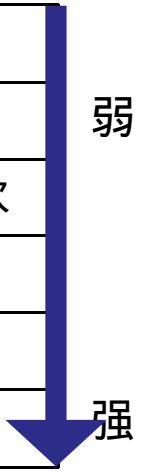
x=1, y=3, z=0　　TF　　sacbd

x=1, y=1, z=1　　FF　　sabd



白盒测试

语句覆盖	每条语句至少执行一次
判定覆盖	程序中每个分支的可能值至少出现一次
条件覆盖	程序中每个判定的每个条件的各种可能值至少出现一次
判定条件覆盖	同时满足判定覆盖标准和条件覆盖标准
条件组合覆盖	每个判定中各条件的每种组合至少出现一次
路径覆盖	程序中的每条可能路径至少执行一次





目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

单元测试工具-go benchmark基准测试

CPU测试（随机数排序）

```
func BenchmarkBubbleSort(t *te
   n := 100
   for i := 1; i < 5; i++ {
      nums := generate(n)
      bubbleSort(nums)
      n *= 10
   }
}
```

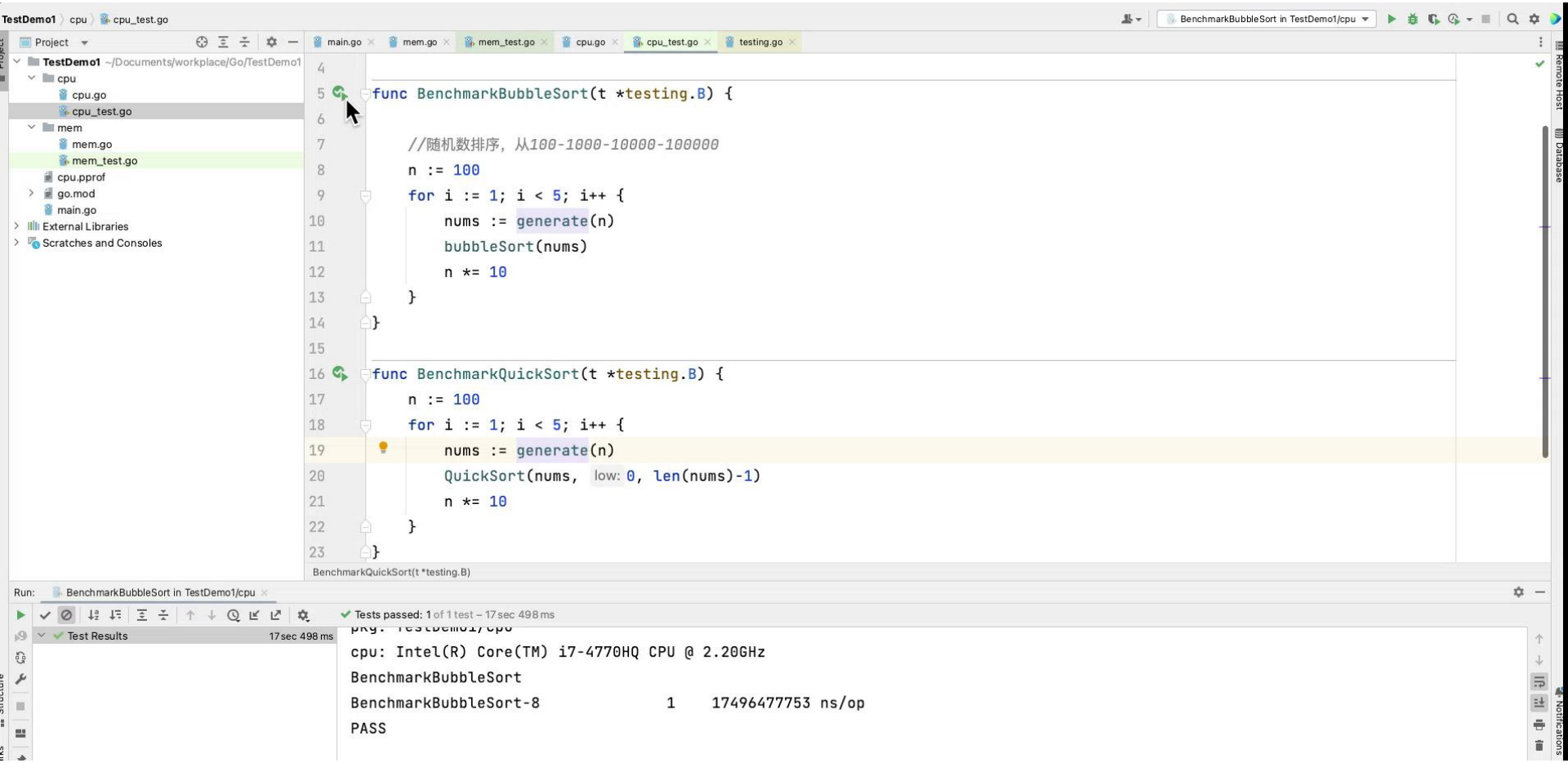
```
func BenchmarkQuickSort(t *testing.B){
   n := 100
   for i := 1; i < 5; i++ {
      nums := generate(n)
      QuickSort(nums, 0, len(nums)-1)
      n *= 10
   }
}
```

内存测试

```
//拼接字符串（n 个字符）
func BenchmarkConcat(b *testing.B) {
   for i := 1; i < 1000; i++ {
      concat(i)
   }
}
```

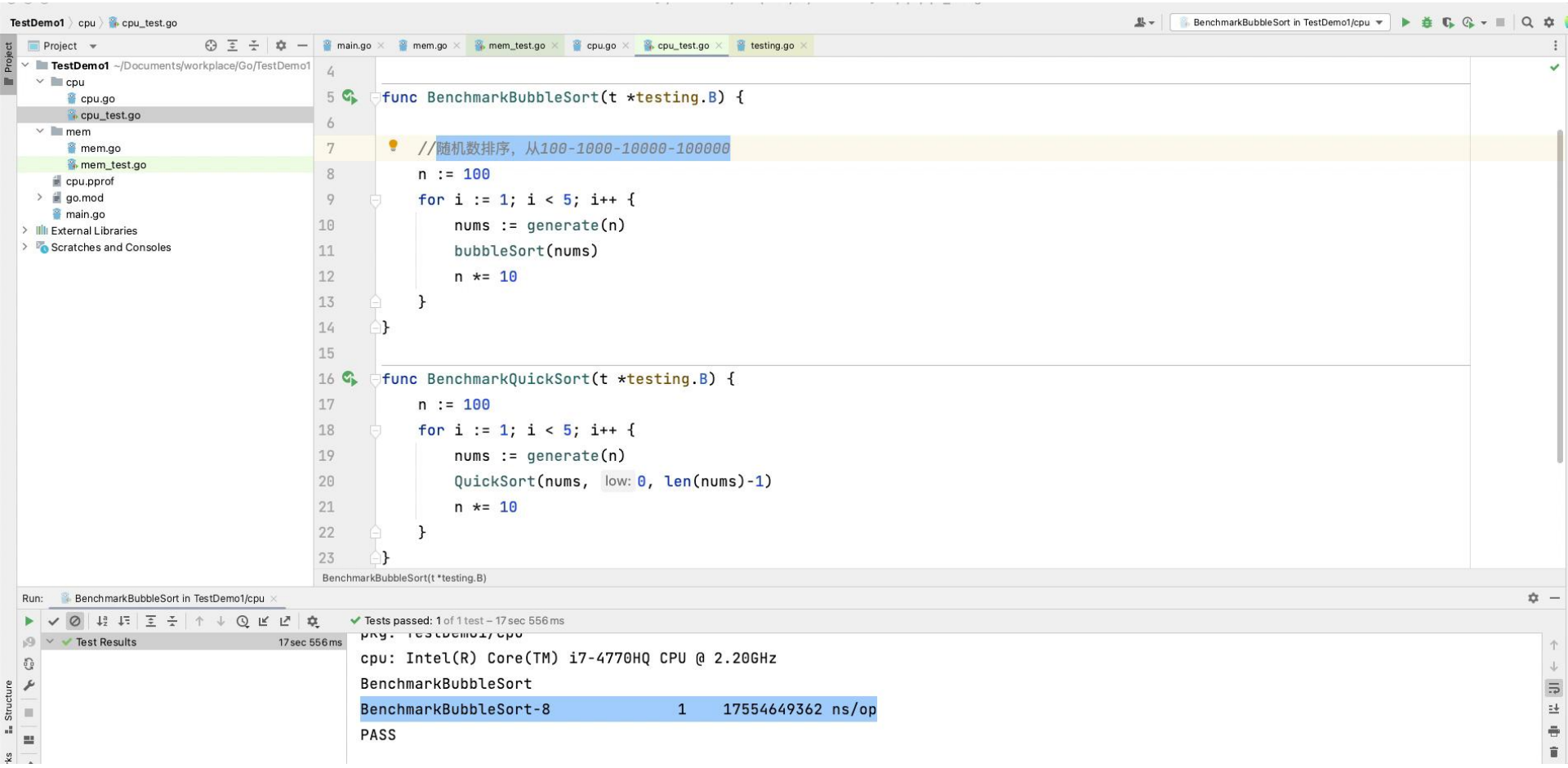
单元测试工具-go benchmark基准测试

CPU测试（冒泡排序）



单元测试工具-go benchmark基准测试

CPU测试（快速排序）





目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

性能测试工具-go pprof

CPU测试

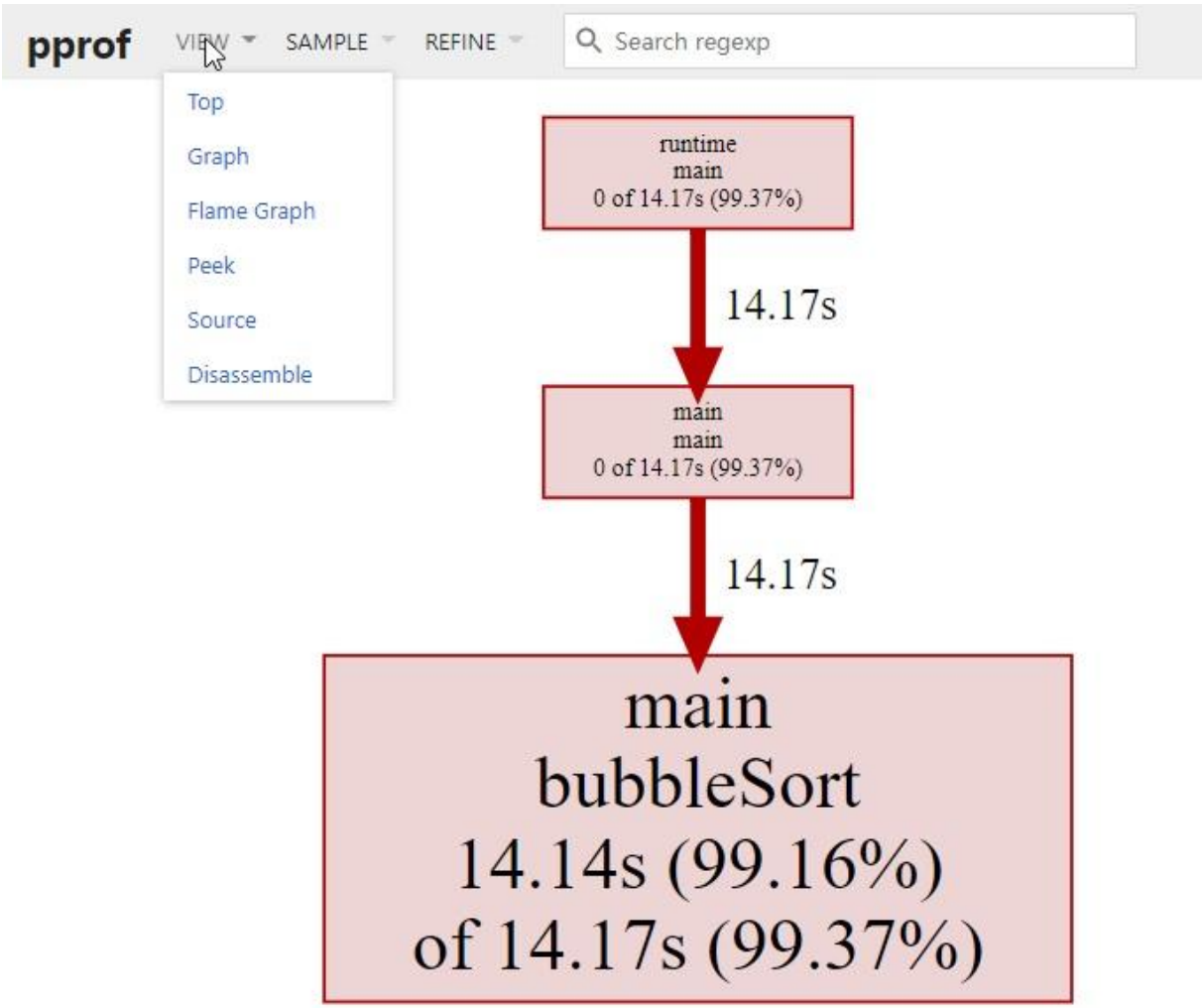
```
f, _ := os.OpenFile("cpu.pprof", os.O_CREATE|os.O_RDWR, 0444)
defer f.Close()
pprof.StartCPUProfile(f)
defer pprof.StopCPUProfile()
```

内存测试

```
defer profile.Start(profile.MemProfile,
profile.MemProfileRate(1)).Stop()
concat(100)
```

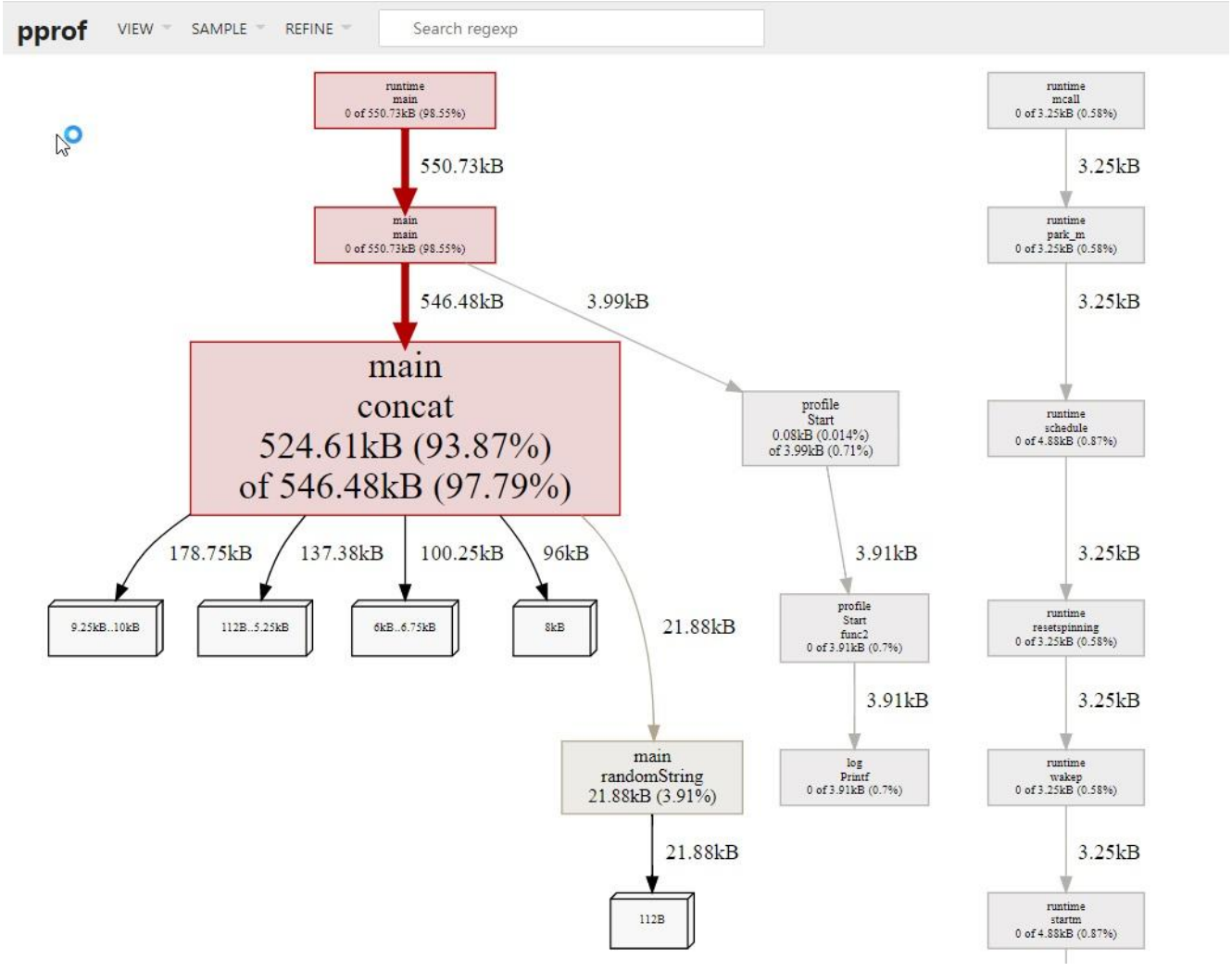
性能测试工具-go pprof

CPU测试



性能测试工具-go pprof

内存测试





目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

UI自动化工具介绍-playwright for python

Playwright是一个强大的Python库，仅用一个API即可自动执行Chromium、Firefox、WebKit等主流浏览器自动化操作，并同时支持以无头模式、有头模式运行。

Playwright提供的自动化技术是绿色、功能强大、可靠且快速，支持Linux、Mac以及Windows操作系统。

1.Playwright安装

```
安装playwright库
pip install playwright
安装浏览器驱动文件（安装过程稍微有点慢）
python -m playwright install
```

Playwright开源地址： https://github.com/microsoft/playwright-python

UI自动化工具介绍-playwright for python

2.Playwright自动测试代码生成

```
python -m playwright codegen --target python -o test.py -b   chromium
https://www.baidu.com
```

3.运行自动测试代码

```
python test.py
```



目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

常见测试工具

白盒测试框架：gtest、boosttest

Googletest（通常简称为gtest）是由Google开发的一个C++单元测试框架，用于编写和运行自动化测试。它支持各种测试方法，包括单元测试、集成测试和功能测试等，并提供了丰富的测试工具和测试报告，以帮助开发人员快速、准确地测试和验证代码的正确性和健壮性。

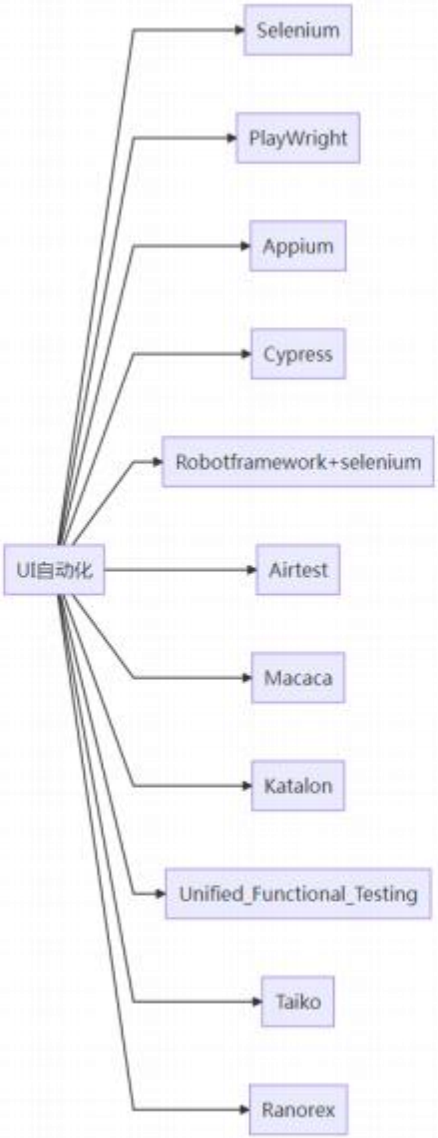
内存测试工具：valgrind

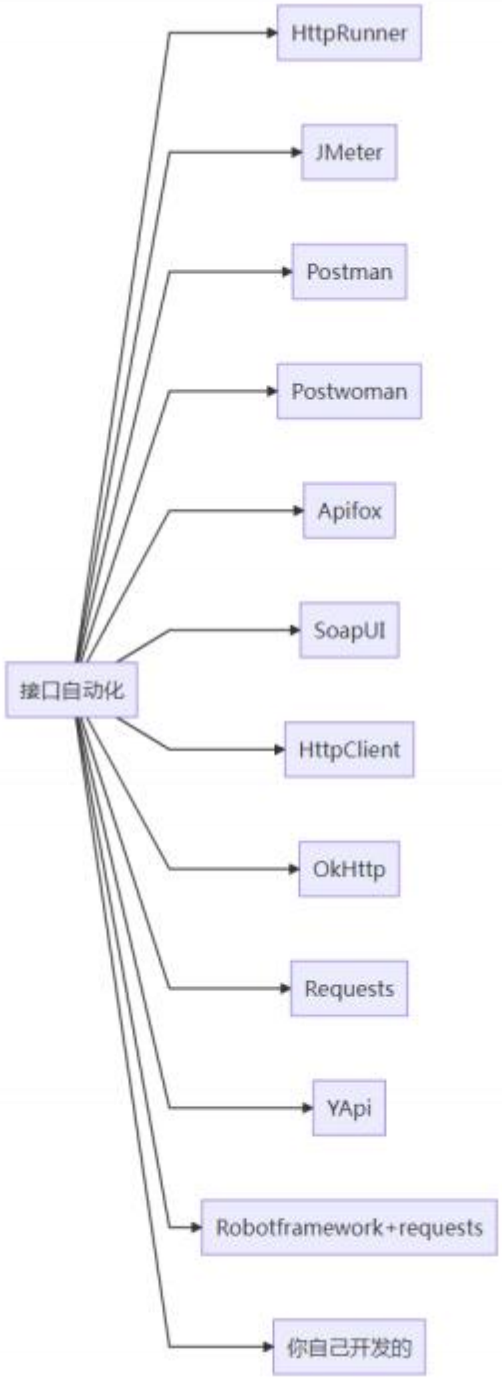
Valgrind是一套Linux下，开放源代码的仿真调试工具的集合。Valgrind由内核以及基于内核的其他调试工具组成。内核模拟了一个CPU环境，并提供服务给其他工具；而其他工具类似于插件，利用内核提供的服务完成各种特定的内存调试任务。

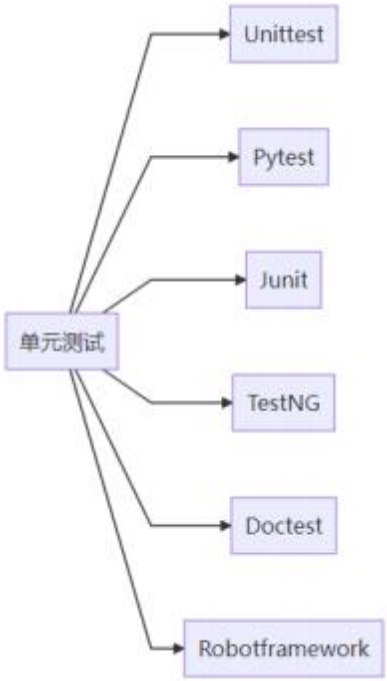
性能测试工具：gprof、perf

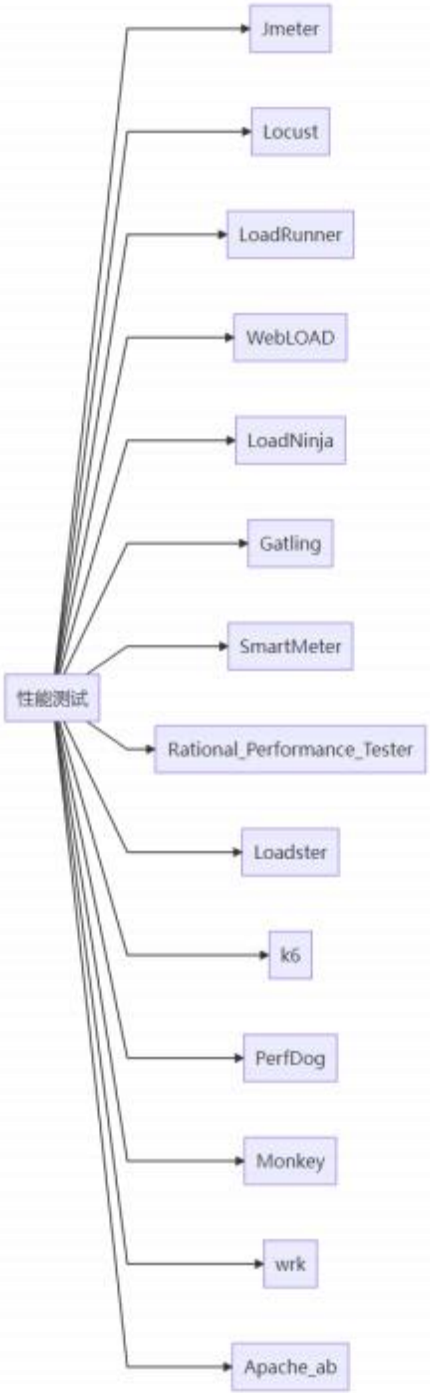
gprof(GNU profiler)是GNU binutils工具集中的一个工具，Linux系统当中会自带这个工具。它可以分析程序的性能，能给出函数调用时间、调用次数和调用关系，找出程序的瓶颈所在。在编译和链接选项中都加入-pg之后，gcc会在每个函数中插入代码片段，用于记录函数间的调用关系和调用次数，并采集函数的调用时间。

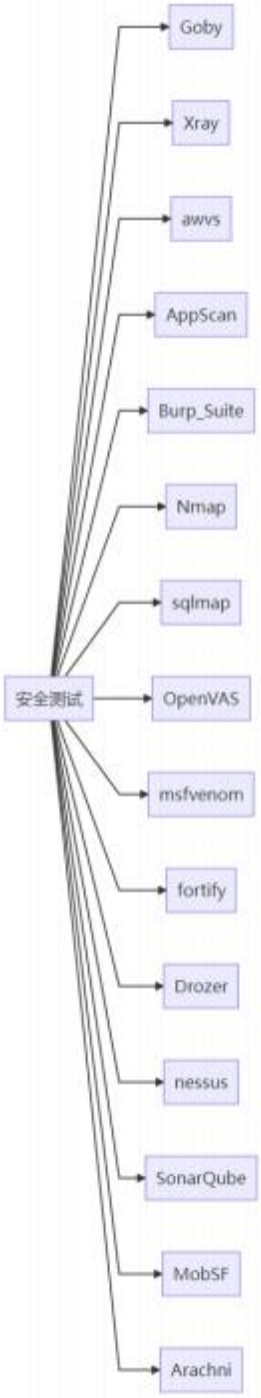
测试工具推荐

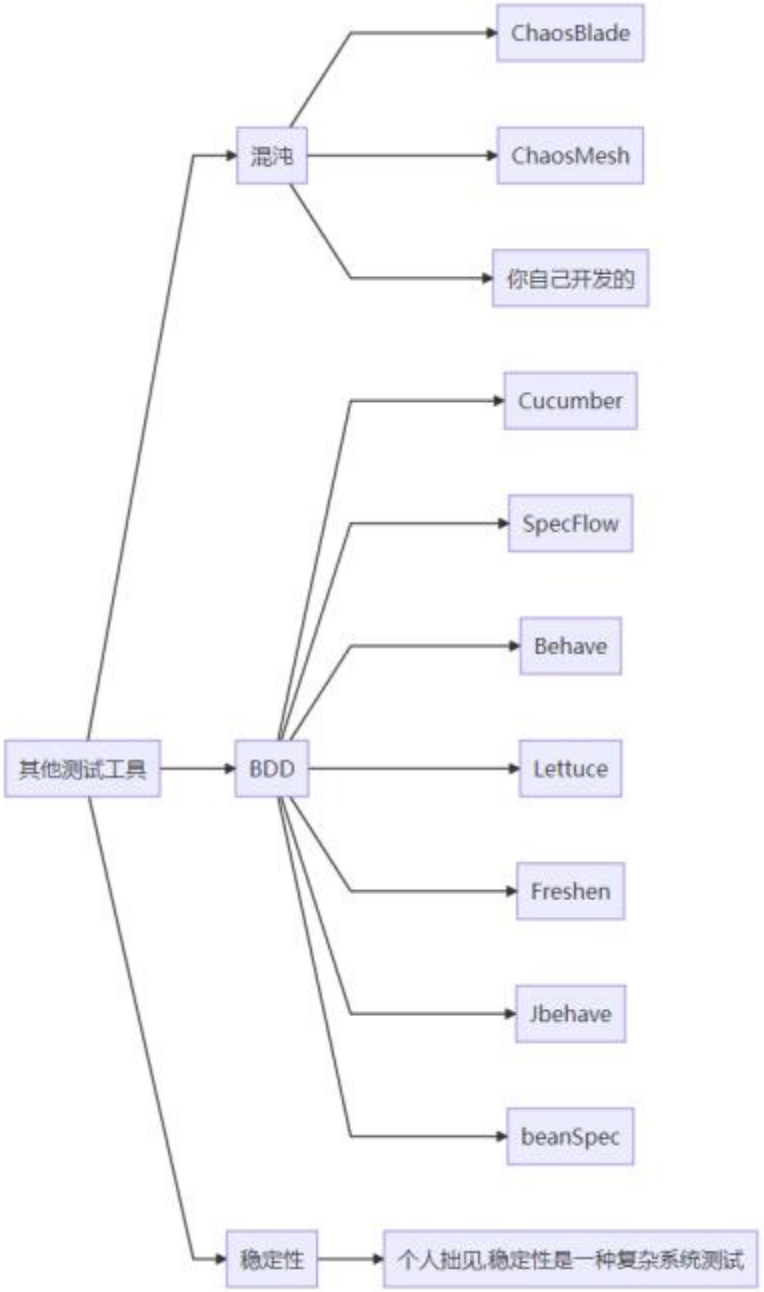

UI自动化
Selenium
PlayWright
Appium
Cypress
Robotframework+selenium
Airtest
Macaca
Katalon
Unified_Functional_Testing
Taiko
Ranorex

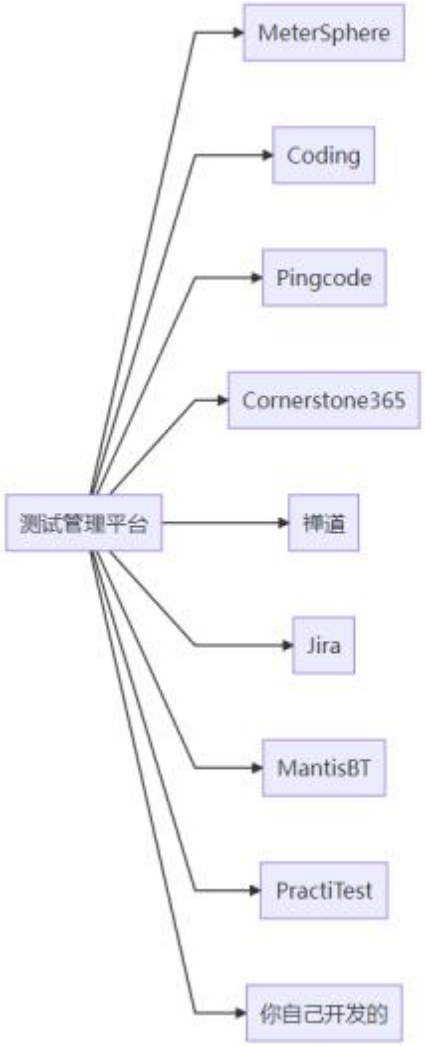

接口自动化
HttpRunner
JMeter
Postman
Postwoman
Apifox
SoapUI
HttpClient
OkHttp
Requests
YApi
Robotframework+requests
你自己开发的


单元测试
Unittest
Pytest
Junit
TestNG
Doctest
Robotframework


性能测试
Jmeter
Locust
LoadRunner
WebLOAD
LoadNinja
Gatling
SmartMeter
Rational_Performance_Tester
Loadster
k6
PerfDog
Monkey
wrk
Apache_ab


安全测试
Goby
Xray
awvs
AppScan
Burp_Suite
Nmap
sqlmap
OpenVAS
msfvenom
fortify
Drozer
nessus
SonarQube
MobSF
Arachni





测试工具推荐:

https://mp.weixin.qq.com/s/Kc0E4Ttx4FkBUSU_IhEPKw



目录

- 软件测试分类
  - 按是否运行分类
  - 按测试步骤分类
  - 按测试方法分类
- 测试工具展示
  - go benchmark
  - go pprof
  - UI自动化工具-playwright
- 其他测试工具介绍
- 实验内容

实验环节

对于选择Linux C++平台进行项目开发的同学，完成本课程实验一：

- 实验内容见《实验指导书1》
- 要求掌握基本的程序编译和调试方法
- 要求各小组构建好自己的开发环境，组内开发环境统一
- 不需要提交实验报告

对于选择Linux C++平台进行项目开发的同学，完成本课程实验四：

- 实验内容见《实验指导书4》
- 要求掌握大型程序的构建方法：make
- 要求学会使用gtest、valgrind、gprof、perf等测试框架和工具
- 不需要提交实验报告

项目要求：

- （推荐）采用gtest测试框架，完成项目中任一模块的白盒单元测试。测试代码留在项目源代码中。
- （推荐）在项目开发过程中，使用valgrind、gprof、perf等工具。在项目报告中体现出来。
- 项目报告中，至少包含两种以上测试类型（单元测试、集成测试、性能测试、稳定性测试等）。
- 项目报告中，至少包含15个以上测试用例。


Thank You